

Healthcare Appointment & Follow-up Manager

Final Implementation Plan

1. Final Technology Stack

We will use:

Layer	Technology	Purpose
Frontend	React + Vite	Patient, Doctor and Admin portals
Backend	Flask	REST API and business logic
ORM	SQLAlchemy	Python ↔ PostgreSQL interaction
Database	PostgreSQL	Users, doctors, appointments, prescriptions, AI outputs, etc.
Authentication	JWT	Login and API authentication
Password Security	Werkzeug/bcrypt	Password hashing
Background Jobs	Celery	Reminders, email retries
Message Broker	Redis	Queue for Celery
LLM	LLM API	Pre/post-visit summaries
Email	SendGrid/Mailgun	Transactional emails
Calendar	Google Calendar API	Appointment calendar events
Calendar Auth	OAuth 2.0	Google Calendar authorization
Frontend Deployment	Vercel	React hosting
Backend Deployment	Render/Railway	Flask hosting
DB Hosting	Managed PostgreSQL	Production database

Important

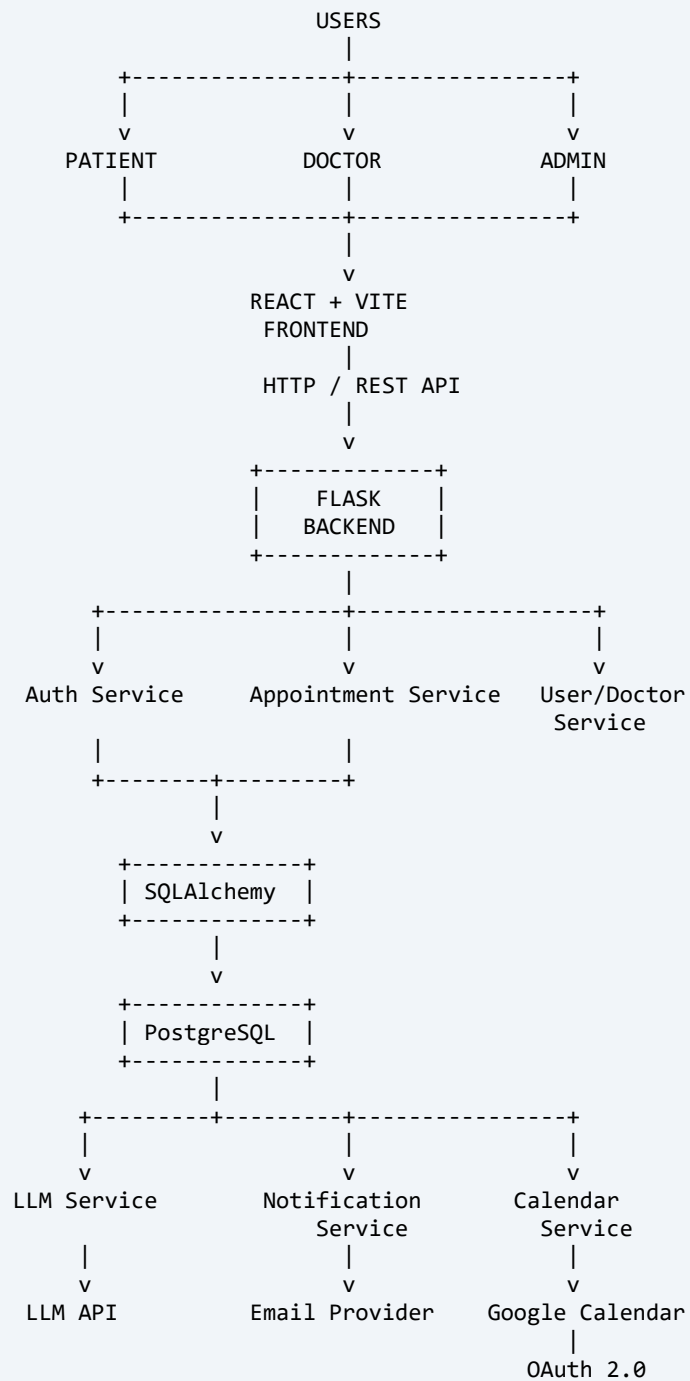
We are not initially adding:

- Microservices
- API Gateway
- Load Balancer
- S3
- Kubernetes
- Docker
- Complex monitoring infrastructure

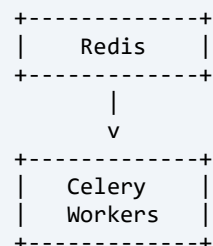
These can be future enhancements if needed.

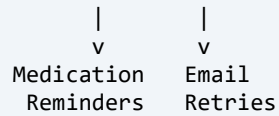
2. Overall Architecture

The final architecture should conceptually be:



BACKGROUND PROCESSING





3. Most Important Design Principle

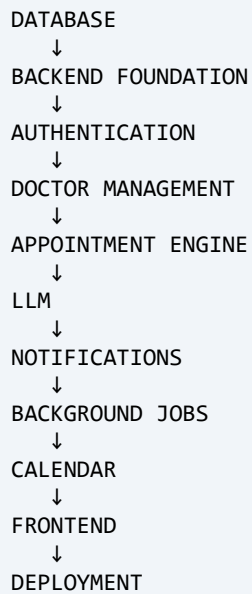
Build the project in layers.

Do NOT do this:

Login
LLM
Calendar
Frontend
Email
Database

randomly.

Instead:



This gives us a stable foundation.

4. Phase 0 — Project Setup

First thing you should do

Create the project directory.

```
healthcare-appointment-manager/  
|
```

```
├── backend/
├── frontend/
├── database/
├── docs/
├── .gitignore
├── README.md
└── .env.example
```

Then initialize Git.

```
healthcare-appointment-manager
├──
└── --- Git repository
```

5. Backend Structure

Inside backend:

```
backend/
├── app/
│   ├── __init__.py
│   ├── config/
│   │   └── settings.py
│   ├── models/
│   │   ├── user.py
│   │   ├── doctor.py
│   │   ├── appointment.py
│   │   ├── symptom.py
│   │   ├── prescription.py
│   │   ├── notification.py
│   │   └── calendar_event.py
│   ├── routes/
│   │   ├── auth_routes.py
│   │   ├── patient_routes.py
│   │   ├── doctor_routes.py
│   │   ├── admin_routes.py
│   │   └── appointment_routes.py
│   ├── services/
│   │   ├── auth_service.py
│   │   ├── appointment_service.py
│   │   ├── doctor_service.py
│   │   ├── llm_service.py
│   │   ├── email_service.py
│   │   └── calendar_service.py
│   ├── tasks/
│   │   ├── medication_tasks.py
│   │   └── email_tasks.py
│   └── utils/
│       ├── decorators.py
│       └── validators.py
└── migrations/
```

```
├─ tests/
├─ requirements.txt
├─ run.py
└─ .env
```

This separation will also make your project easier to explain in interviews.

6. Phase 1 — Database Design

This is where I recommend you start.

Before building React, create the database.

We need to identify our major entities.

User

Doctor

WorkingHours

DoctorLeave

Appointment

Symptom

PreVisitSummary

VisitNotes

Prescription

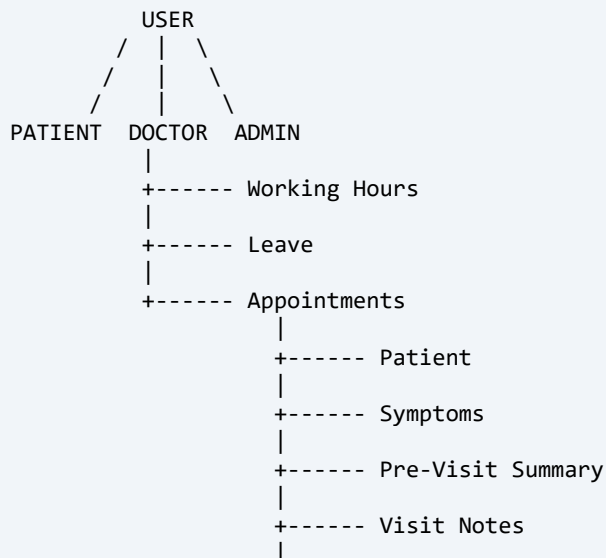
Medication

Notification

CalendarEvent

7. Basic Entity Relationships

Conceptually:



```
+----- Prescription
|
+----- Calendar Event
|
+----- Notifications
```

8. Important Database Decision

We should not simply create a table containing every possible time slot.

Instead, we can derive available slots from:

```
Doctor Working Hours
+
Slot Duration
+
Doctor Leave
+
Existing Appointments
+
Slot Hold
=
Available Slots
```

This keeps the appointment model cleaner.

9. Appointment Model

The appointment should contain things such as:

```
appointment_id
patient_id
doctor_id
appointment_date
start_time
end_time
status
created_at
updated_at
```

Status could be:

```
HELD
CONFIRMED
CANCELLED
COMPLETED
RESCHEDULED
```

The exact model will be finalized during database implementation.

10. Double-Booking Prevention

This is one of the most important parts of the entire project.

Suppose:

```
Patient A ---> 10:00 AM
Patient B ---> 10:00 AM
```

Both click Book simultaneously.

We cannot rely only on:

```
"Check if slot is available"
```

because both requests could check at almost the same time.

Instead:

```
Request A ----- \
                    \ DATABASE TRANSACTION
Request B ----- /
```

We will use:

```
Database transaction
+
appropriate locking/constraint strategy
```

to guarantee that only one booking succeeds.

This should be designed before implementing the booking API.

11. Slot Hold Mechanism

The assignment explicitly asks us to address slot holds.

Conceptually:

```
Patient selects slot
      |
      v
    HOLD SLOT
      |
      |---- 5/10 minutes ----|
      |
      v
Patient confirms?
  /      \
YES       NO
  |        |
  v        v
CONFIRMED  RELEASE
```

The exact hold duration is a design decision.

The important point is:

A temporary hold prevents multiple users from completing the same booking while one user is in the confirmation process.

12. Phase 2 — Flask Backend Foundation

Once the database design is ready:

Create:

```
Flask application
|
+-- Configuration
+-- Database connection
+-- SQLAlchemy
+-- Error handling
+-- Environment variables
```

First make sure:

```
Flask
|
SQLAlchemy
|
PostgreSQL
```

works correctly.

13. Phase 3 — Authentication

Implement authentication before the portals.

Order:

```
Register
↓
Login
↓
Password Hashing
↓
JWT Generation
↓
JWT Verification
↓
Role Authorization
```

Roles:

PATIENT

DOCTOR

ADMIN

Then enforce:

Patient → patient APIs
Doctor → doctor APIs
Admin → admin APIs

14. Phase 4 — Admin Module

Build the Admin functionality next.

Admin can:

Create Doctor
Update Doctor
View Doctor
Manage Specialisation
Set Working Hours
Set Slot Duration
Set Leave Days

Logic:

```
ADMIN
|
v
Doctor Management
|
+-- Profile
+-- Specialisation
+-- Working Hours
+-- Slot Duration
+-- Leave
```

15. Phase 5 — Doctor Availability Engine

Before booking, we need to know:

"Which slots are actually available?"

The logic becomes:

```
Doctor Working Hours
|
v
Generate Possible Slots
|
v
Remove Leave Dates
|
v
Remove Existing Appointments
|
v
Remove Active Holds
```

|
v
AVAILABLE SLOTS

Example:

Working hours:
09:00 - 12:00

Slot duration:
30 minutes

Possible:

09:00
09:30
10:00
10:30
11:00
11:30

Then remove unavailable slots.

16. Phase 6 — Appointment Module

Now implement:

Search Doctor
↓
View Availability
↓
Select Slot
↓
Enter Symptoms
↓
Hold Slot
↓
Confirm
↓
Create Appointment

APIs will roughly be:

```
GET /doctors
GET /doctors/{id}
GET /doctors/{id}/availability

POST /appointments
GET /appointments
GET /appointments/{id}

PUT /appointments/{id}/reschedule
PUT /appointments/{id}/cancel
```

Exact endpoint naming can be finalized during implementation.

17. Phase 7 — Doctor Leave Conflict

This needs special handling.

Suppose:

```
Doctor has appointment:

Monday 10:00 → Patient A
Monday 11:00 → Patient B
```

Admin now marks:

```
Monday = LEAVE
```

System:

```
Leave Created
  |
  v
Find affected appointments
  |
  v
Cancel / mark affected appointments
  |
  v
Notify patients
  |
  v
Update Calendar
```

This logic should be transactional where appropriate.

18. Phase 8 — Symptom Module

Patient enters:

```
Symptoms
```

before final appointment confirmation.

Store:

```
Appointment
  |
  +-- Symptoms
```

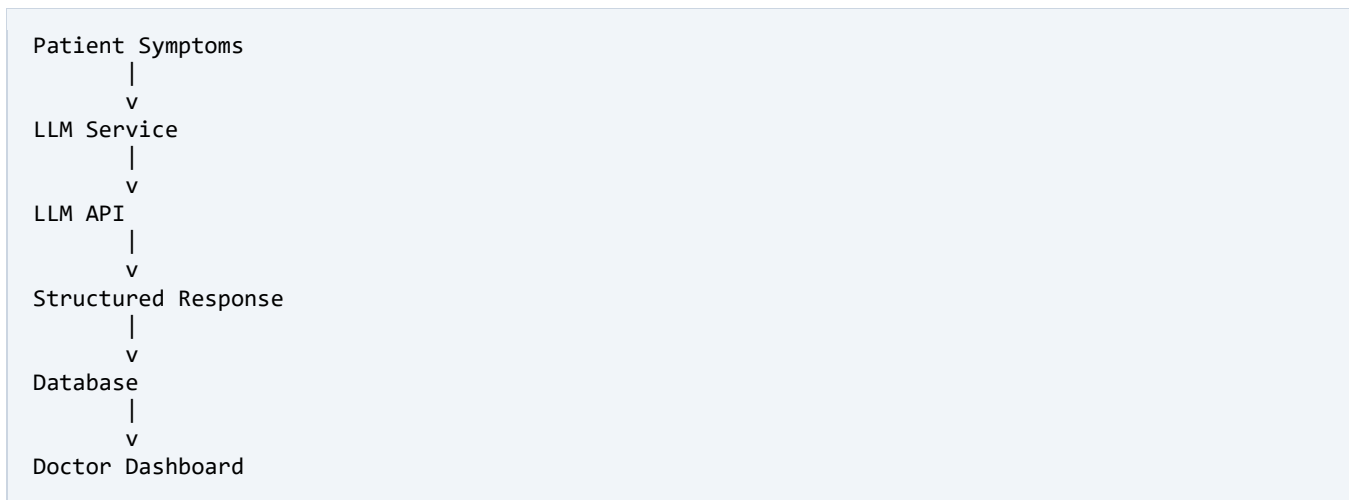
Then send symptoms to the LLM.

19. Phase 9 — Pre-Visit LLM

Create a dedicated:

```
llm_service.py
```

Flow:



Required output:

Urgency
Chief Complaint
Question 1
Question 2
Question 3

20. Important LLM Design

The LLM should not be treated as the source of medical truth.

Its purpose is:

Summarization
Organization
Communication assistance

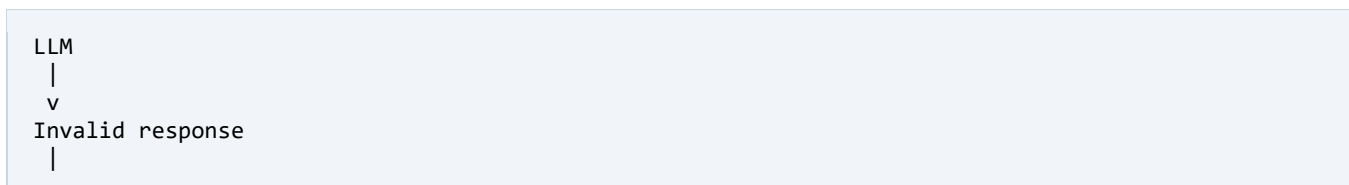
Not:

Diagnosis
Treatment decision
Emergency medical decision

Therefore, the doctor remains responsible for clinical decisions.

Also, validate the LLM response.

If it returns something malformed:



v
Fallback

The appointment should still work.

21. Phase 10 — Doctor Consultation Module

Doctor dashboard:

```
Doctor
|
v
Today's Appointments
|
v
Select Patient
|
+-- Symptoms
+-- AI Pre-Visit Summary
|
v
Consultation
|
+-- Notes
+-- Prescription
```

Doctor submits the information.

22. Phase 11 — Post-Visit LLM

Flow:

```
Doctor Notes
+
Prescription
|
v
LLM Service
|
v
Patient-Friendly Summary
|
v
Database
|
v
Patient Portal
```

The patient should receive an understandable summary.

23. Phase 12 — Medication Module

Prescription contains something like:

Medicine

Dosage

Frequency

Duration

Example:

```
Medicine: X
Frequency: Twice daily
Duration: 5 days
```

The system creates reminder tasks.

```
Prescription
  |
  v
Medication Schedule
  |
  v
Celery
  |
  v
Reminder
```

24. Phase 13 — Email Notification Module

Create:

```
email_service.py
```

Events that trigger emails:

- Appointment booked
- Appointment reminder
- Appointment cancelled
- Doctor leave
- Appointment rescheduled
- Medication reminder

Architecture:

```
Application Event
  |
  v
Notification Task
  |
  v
Celery
  |
  v
Email Provider
  |
  v
Patient / Doctor
```

25. Email Failure Handling

Suppose:

```
Send Email
  |
  X
Failure
```

Don't lose the notification.

Instead:

```
Failure
  |
  v
Retry
  |
  v
Retry again
  |
  v
Success
```

or eventually:

```
Retry exhausted
  |
  v
Log failure
  |
  v
Notification marked FAILED
```

This directly addresses the assignment's notification reliability requirement.

26. Phase 14 — Celery + Redis

Only after the core synchronous application works should we introduce background processing.

Use Celery for:

Medication reminders

Email retries

Scheduled appointment reminders

Architecture:

```
Flask
  |
  v
Redis Queue
  |
  v
Celery Worker
  |
```



```
+---- Email
|
+---- Reminder
```

27. Phase 15 — Google Calendar

Now implement calendar integration.

Create:

```
calendar_service.py
```

Flow:

```
Appointment Confirmed
  |
  v
Google OAuth
  |
  v
Google Calendar API
  |
  v
Create Event
```

Store the external event ID:

```
appointment
  |
  +-- google_calendar_event_id
```

This is important because later:

```
Reschedule
  |
  v
Find Google Event ID
  |
  v
Update Event
```

Cancellation:

```
Cancel
  |
  v
Google Event ID
  |
  v
Delete Event
```

28. Phase 16 — React Frontend

Only after the core backend APIs are working should we build the complete frontend.

Create:

```
frontend/  
├── src/  
│   ├── components/  
│   ├── pages/  
│   ├── services/  
│   ├── context/  
│   ├── hooks/  
│   └── utils/
```

29. Patient Portal

Patient screens:

- Login
- Register
- Dashboard
- Doctor Search
- Doctor Profile
- Available Slots
- Symptom Form
- Booking Confirmation
- My Appointments
- Appointment Details
- Reschedule
- Cancel
- Post-Visit Summary
- Prescription
- Medication Reminders

30. Doctor Portal

Doctor screens:

- Login
- Dashboard
- Today's Appointments
- Appointment Details
- Patient Symptoms
- AI Pre-Visit Summary
- Consultation Notes
- Prescription
- Post-Visit Summary
- Appointment History

31. Admin Portal

Admin screens:

Login
Dashboard
Doctors
Create Doctor
Edit Doctor
Working Hours
Slot Duration
Leave Management
Appointments

32. Phase 17 — Testing

Testing should not be an afterthought.

We specifically need to test:

Authentication

Correct login
Wrong password
Unauthorized API
Wrong role

Appointment

Available slot
Unavailable slot
Cancellation
Rescheduling
Double booking

Most important:

```
Patient A ----> Same Slot  
Patient B ----> Same Slot
```

```
Expected:  
Only ONE succeeds
```

Leave

Leave + existing appointment

LLM

LLM success
LLM timeout
LLM malformed response
LLM unavailable

Email

Success
Failure
Retry

Calendar

Create
Update
Delete
OAuth failure

33. Phase 18 — Deployment

Once everything works locally:

```
React
  |
  v
Vercel

Flask
  |
  v
Render / Railway

PostgreSQL
  |
  v
Managed PostgreSQL

Redis
  |
  v
Managed Redis
```

Configure environment variables.

Never commit:

.env
API keys
JWT secret
Google client secret
LLM API key
Email API key
Database password

34. Phase 19 — Documentation

Finally create:

```
README.md
```

containing:

Project Overview
Features
Architecture
Tech Stack
Installation
Environment Variables
Database Setup
Backend Setup
Frontend Setup
API Documentation
LLM Prompts
Google OAuth Setup
Celery/Redis Setup
Deployment
Testing
Known Limitations

Also prepare:

```
docs/  
├── architecture.md  
├── database-schema.md  
├── api-documentation.md  
└── system-design.md
```

35. Final Module Breakdown

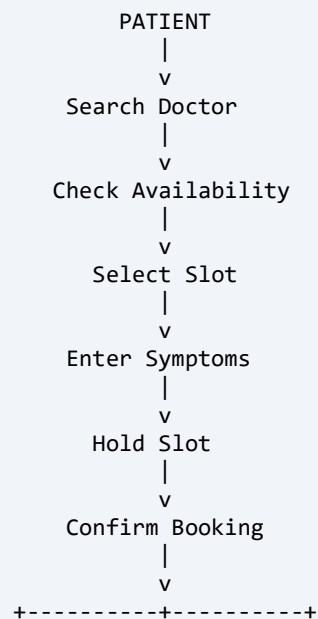
Our application should have these major modules:

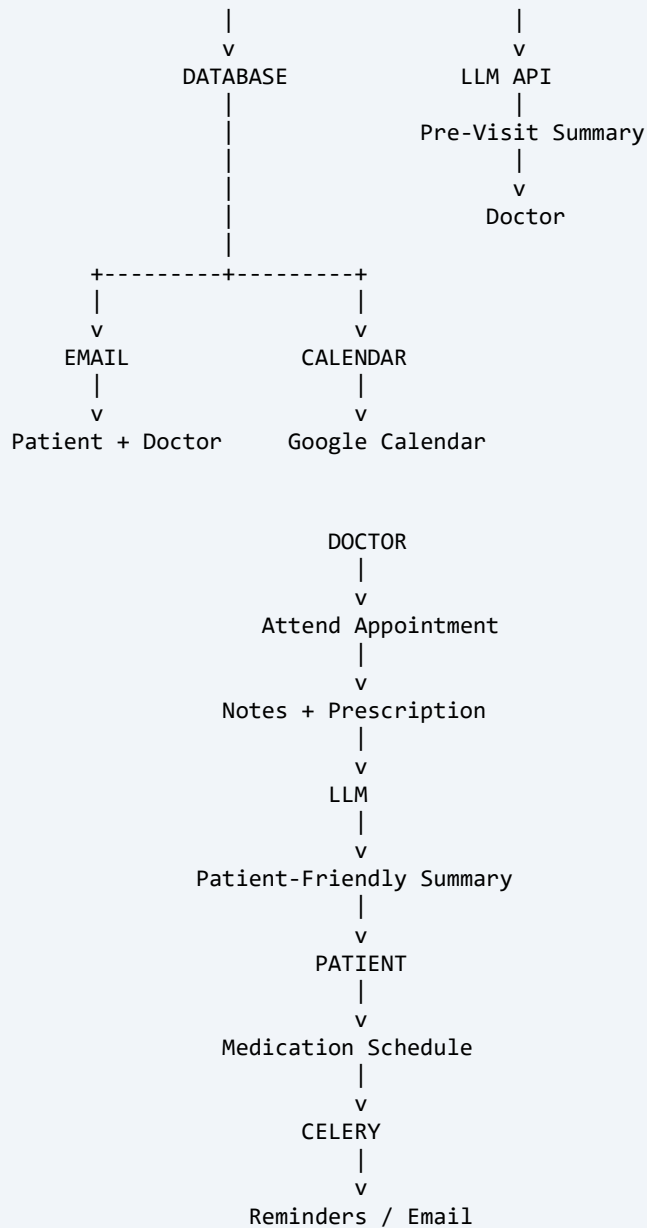
```
Healthcare Appointment Manager  
├── Authentication Module  
├── User Module  
├── Patient Module  
└── Doctor Module
```

- Admin Module
- Working Hours Module
- Doctor Leave Module
- Availability Module
- Slot Hold Module
- Appointment Module
- Symptom Module
- Pre-Visit AI Module
- Consultation Module
- Prescription Module
- Post-Visit AI Module
- Medication Reminder Module
- Notification Module
- Email Module
- Google Calendar Module
- Background Job Module
- Database Module

36. Overall Business Logic

The entire application can be understood as this:





37. What Should You Start With TODAY?

This is the most important part.

Don't start with React.

Don't start with the LLM.

Don't start with Google Calendar.

Don't start with Celery.

Start here:

STEP 1

```
Create project repository
↓
STEP 2
Create PostgreSQL database
↓
STEP 3
Design database schema
↓
STEP 4
Create Flask application
↓
STEP 5
Connect Flask → SQLAlchemy → PostgreSQL
↓
STEP 6
Create database models
↓
STEP 7
Implement authentication
↓
STEP 8
Implement doctor management
↓
STEP 9
Implement availability/slot generation
↓
STEP 10
Implement slot hold + double-booking protection
↓
STEP 11
Implement appointment booking
```

Stop here and make sure the core booking system is solid.

Then:

```
STEP 12
Doctor leave conflict handling
↓
STEP 13
Patient symptom form
↓
STEP 14
Pre-visit LLM
↓
STEP 15
Doctor consultation
↓
STEP 16
Prescription
↓
STEP 17
Post-visit LLM
↓
STEP 18
Celery + Redis
↓
STEP 19
Medication reminders
↓
STEP 20
Email + retries
```


↓
STEP 21
Google Calendar
↓
STEP 22
React portals
↓
STEP 23
Testing
↓
STEP 24
Deployment
↓
STEP 25
README + system design

38. Our Development Strategy

I recommend we work on this one phase at a time.

For each phase, I'll help you with:

Requirements
↓
Design
↓
Folder structure
↓
Database/API design
↓
Implementation
↓
Testing
↓
Interview explanation

And importantly, we won't add unnecessary technologies just to make the architecture look impressive.

The evaluator is specifically looking for whether you can solve:

CORE PROBLEMS
Double Booking
Simultaneous Requests
Slot Holds
Doctor Leave Conflicts
Notification Failures
LLM Failures
Database Design
Calendar Synchronization

Those are where we should spend our engineering effort.

Your first actual implementation task

Start with the database design.

Specifically, our next step should be:

1. Identify every entity
2. Define every table
3. Define columns and data types
4. Define primary/foreign keys
5. Define relationships
6. Define appointment statuses
7. Design slot-hold mechanism
8. Design double-booking protection
9. Decide which constraints belong in PostgreSQL
10. Create the initial PostgreSQL schema

Once that is finalized, then we build the Flask backend around it. This gives us the strongest foundation for the entire project.